Lösungshinweise/-vorschläge zu
Auswertung: Beispiel Übungsblatt

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Aufgabe 1. Entwurfsmuster

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

(a) Schreiben Sie eine Funktion `replicate: 'a -> Nat -> 'a list`, die einen beliebigen Wert `x` sowie eine natürliche Zahl `n` nimmt und eine Liste zurückgibt, die `n` mal das Element `x` enthält.

Beispiele:

```
replicate 0N 3N = [0N; 0N; 0N]
replicate 3N 0N = []
replicate "Hallo" 2N = ["Hallo"; "Hallo"]
```

(i) Gehen Sie **strikt nach dem Peano Entwurfsmuster** vor.

Original

```
let rec replicatePeano (x: 'a) (n: Nat): 'a list =
    if n = 1N then [x]
    else x :: replicatePeano x (n - 1N)
```

Lösung

```
let rec replicatePeano (x: 'a) (n: Nat): 'a list =
    if n = 0N then []
    else x :: replicatePeano x (n - 1N)
```

(ii) Gehen Sie **strikt nach dem Leibniz Entwurfsmuster** vor.

Original

```
let rec replicateLeibniz (x: 'a) (n: Nat): 'a list =
    if n = 0N then []
    else
        let xs: 'a list = replicateLeibniz x (n / 2N)
        if n % 2N = 0N then x :: (xs @ xs)
        else x :: x :: (xs @ xs)
```

Lösung

```
let rec replicateLeibniz (x: 'a) (n: Nat): 'a list =
    if n = 0N then []
    else
        let xs: 'a list = replicateLeibniz x (n / 2N)
        if n % 2N = 0N then xs @ xs
        else x :: (xs @ xs)
```

(b) Schreiben Sie eine Funktion `exists`, die eine Funktion `f` vom Typ `Nat -> bool` sowie zwei natürliche Zahlen `lower` und `upper` nimmt und prüft, ob es eine Zahl `x` im Intervall `lower..upper` gibt, für die `f x = true` ist.

Beispiele:

```
exists (fun x -> x = 9N) 10N 20N = false
exists (fun x -> x = 10N) 10N 20N = true
exists (fun x -> x = 15N) 10N 20N = true
exists (fun x -> x = 20N) 10N 20N = true
exists (fun x -> x = 21N) 10N 20N = false
exists (fun x -> true) 20N 10N = false
exists (fun x -> x = 42N) 42N 42N = true
```

```
let rec exists (f: Nat -> bool) (lower: Nat) (upper: Nat): bool =
    lower < upper && (f lower || exists f (lower + 1N) upper)
```

OG

```
module Solution


open Types


//%%%
```

(c) Schreiben Sie eine rekursive Funktion `mod7: Nat -> Nat`, die für eine gegebene Zahl `n` den Rest der Ganzzahldivision durch `7` (also `n` modulo `7`) berechnet. Gehen Sie strikt nach dem Peano-Entwurfsmuster vor, verwenden Sie nicht die Operationen `%` oder `/`.

Beispiele:

```
mod7 0N = 0N        mod7 3N = 3N        mod7 6N = 6N
mod7 1N = 1N        mod7 4N = 4N        mod7 7N = 0N
mod7 2N = 2N        mod7 5N = 5N        mod7 8N = 1N
```

Original

```
let rec mod7 (n: Nat): Nat =
    if n = 0N then 0N
    else
        let m: Nat = mod7 (n - 1N)
        if m = 7N then 0N
        else m + 1N
```

Lösung

```
let rec mod7 (n: Nat): Nat =
    if n = 0N then 0N
    else
        let m: Nat = mod7 (n - 1N)
        if m = 6N then 0N
        else m + 1N
```

TODO TODO

Beispiele:

```
mod5 0N = 0N      mod5 2N = 2N      mod5 4N = 4N      mod5 19N = 4N     mod5 21N = 1N
mod5 1N = 1N      mod5 3N = 3N      mod5 5N = 0N      mod5 20N = 0N     mod5 22N = 2N
```

```
let f a b =
    let result = a + b
    result
```

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

- A
- B
- C

Lösung

Aufgabe 2. Arrays

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

```
type Item<'a> =
    {
        mutable elem: 'a
        mutable next: option<Item<'a>>
        mutable lst: 'a list
    }
```

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

```
a) digitSum Random (74ms): Error Message: Test method Tests+Tests.a) digitSum Random threw exception:
    System.Exception: Falsifiable, after 1 test (1 shrink) (StdGen (1765833463, 297438798)):
    Original:
    11N
    Shrunk:
    0N
    with exception:
    Microsoft.VisualStudio.TestTools.UnitTesting.AssertFailedException: Assert.AreEqual failed. Expected:<0>. Actual:<1>.
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.ThrowAssertFailed(String assertionName, String message) in /_/src/TestFramework/TestFramework/Assertions/Assert.cs:line 60
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.AreEqual[T](T expected, T actual, IEqualityComparer`1 comparer, String message, Object[] parameters) in /_/src/TestFramework/TestFramework/Assertions/Assert.AreEqual.cs:line 201
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.AreEqual[T](T expected, T actual) in /_/src/TestFramework/TestFramework/Assertions/Assert.AreEqual.cs:line 34
        at Tests.a) digitSum Random@69.Invoke(Nat n) in path\to\Tests.fs:line 69
        at FsCheck.Testable.evaluate[a,b](FSharpFunc`2 body, a a)

    Stack Trace:
        at Microsoft.FSharp.Core.PrintfModule.PrintFormatToStringThenFail@1448.Invoke(String message)
        at <StartupCode$FsCheck>.$Runner.get_throwingRunner@382.FsCheck.IRunner.OnFinished(String name, TestResult testResult)
        at FsCheck.Runner.runner[a](Config config, a prop)
        at FsCheck.Runner.check[a](Config config, a p)
        at FsCheck.Check.One[Testable](Config config, Testable property)
        at Tests.Tests.a) digitSum Random() in path\to\Tests.fs:line 68
        at System.RuntimeMethodHandle.InvokeMethod(Object target, Void** arguments, Signature sig, Boolean isConstructor)
        at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj, BindingFlags invokeAttr)
```

Error MEssage